4

抽象：进程

本章讨论操作系统提供给用户的最基本的抽象之一：**进程**（process）。非正式地说，进程的定义非常简单：它就是一个**运行中的程序**（running program）[V+65, BH70]。程序本身是没有生命的东西：它只是躺在磁盘上，是一堆指令（也许还有一些静态数据），等着被调动起来。正是操作系统拿起这些字节，让它们运行起来，把程序变成有用的东西。

事实上，人们常常想同时运行不止一个程序；比如，想想你的台式机或笔记本电脑，你可能想同时运行网页浏览器、邮件程序、游戏、音乐播放器等等。实际上，一个典型的系统看上去可能同时运行着几十甚至上百个进程。这样一来，系统用起来就很方便，因为你永远不必操心有没有空闲的 CPU；你只管运行程序就行。于是，我们的挑战是：

关键问题：
如何提供有许多 CPU 的假象？
尽管只有少数几个物理 CPU 可用，操作系统怎样才能提供 CPU 几乎取之不尽的假象？

操作系统通过**虚拟化**（virtualizing）CPU 来营造这个假象。它运行一个进程，然后停下它、运行另一个进程，如此往复，从而让人觉得存在许多虚拟 CPU，而实际上只有一个（或少数几个）物理 CPU。这项基本技术叫作 CPU 的**分时**（time sharing）共享，它让用户想运行多少个并发进程就运行多少个；可能的代价是性能，因为如果 CPU 必须被共享，每个进程都会运行得更慢一些。

要实现 CPU 的虚拟化，而且要实现得好，操作系统既需要一些底层的机制，也需要一些高层的智能。我们把底层的机制称为**机制**（mechanism）；机制是实现某项所需功能的底层方法或协议。例如，后面我们会学习如何实现**上下文切换**（context switch），它让操作系统能够在给定的 CPU 上停止运行一个程序、开始运行另一个程序；所有现代操作系统都采用这种分时机制。

在这些机制之上，存放着操作系统的部分智能，其形式是**策略**（policy）。策略是操作系统中用来做出某种决定的算法。例如，有许多程序都可以在 CPU 上运行，操作系统应该运行哪一个？操作系统中的**调度策略**（scheduling policy）会做出这个决定，它很可能会利用历史信息（例如，在过去一分钟里哪个程序运行得更多？）、工作负载知识（例如，运行的是哪些类型的程序？）以及性能指标（例如，系统优化的是交互性能还是吞吐量？）来做决定。

提示：使用分时（和空分）共享

分时共享（time sharing）是操作系统共享资源的一项基本技术。让一个实体使用资源一小会儿，再让另一个实体使用一小会儿，如此往复，这个资源（例如 CPU 或网络链路）就能被许多实体共享。与分时共享相对的是**空分共享**（space sharing），即把资源（在空间上）划分给想要使用它的各方。例如，磁盘空间天然就是一种空分共享的资源；一个块一旦分配给某个文件，通常在用户删除原文件之前，不会再分配给别的文件。

4.1 抽象：进程

操作系统为运行中的程序提供的抽象，我们称之为**进程**（process）。前面说过，进程就是一个运行中的程序；在任何时刻，我们都可以把进程在执行过程中访问或影响的系统各部分清点一遍，以此来概括一个进程。

因此，要理解进程由什么构成，就得理解它的**机器状态**（machine state）：程序在运行时能读取或更新的东西。在任何时刻，机器的哪些部分对这个程序的执行是重要的？

构成进程的机器状态中，一个显而易见的组成部分是它的*内存*。指令在内存中；运行中的程序读写的数据也在内存中。因此，进程能够寻址的内存（称为它的**地址空间**，address space）是进程的一部分。

进程的机器状态还包括*寄存器*；许多指令会显式地读取或更新寄存器，因此寄存器显然对进程的执行很重要。

注意，有一些特别的寄存器也是机器状态的一部分。例如，**程序计数器**（program counter，**PC**，有时也叫**指令指针**，instruction pointer，**IP**）告诉我们程序接下来要执行哪条指令；类似地，**栈指针**（stack pointer）和相关的**帧指针**（frame pointer）用来管理存放函数参数、局部变量和返回地址的栈。

最后，程序也经常访问持久存储设备。这类 *I/O* 信息可能包括进程当前打开的文件列表。

4.2 进程 API

尽管我们要到后面的章节才讨论真正的进程 API，这里先大致介绍一下任何操作系统的接口都必须包含哪些内容。这些 API 以某种形式存在于每一个现代操作系统中。

提示：策略与机制分离

在许多操作系统中，一种常见的设计范式是把高层的策略与底层的机制分离开来[L+75]。你可以把机制看作回答关于系统的*怎么做*的问题；例如，操作系统怎样执行上下文切换？策略则回答选*哪个*的问题；例如，操作系统现在应该运行哪个进程？把两者分开，就能轻松地改变策略，而不必重新考虑机制，因此这是**模块化**（modularity）的一种形式，而模块化是一条通用的软件设计原则。

 © 2008–23, ARPACI-DUSSEAU

- **创建**（create）：操作系统必须提供某种创建新进程的方法。当你在 shell 中输入一条命令，或者双击一个应用程序图标时，就会调用操作系统来创建一个新进程，运行你指定的程序。
- **销毁**（destroy）：既然有创建进程的接口，系统也会提供强行销毁进程的接口。当然，许多进程运行完后会自行退出；但如果它们不退出，用户可能想杀掉它们，因此用来终止失控进程的接口非常有用。
- **等待**（wait）：有时，等待一个进程停止运行是有用的；因此通常会提供某种等待接口。
- **其他控制**（miscellaneous control）：除了杀掉或等待进程，有时还可以对进程进行其他控制。例如，大多数操作系统都提供某种方法来暂停进程（让它停止运行一段时间），之后再恢复它（让它继续运行）。
- **状态**（status）：通常还有一些接口，用来获取进程的状态信息，比如它已经运行了多长时间，或者它处于什么状态。

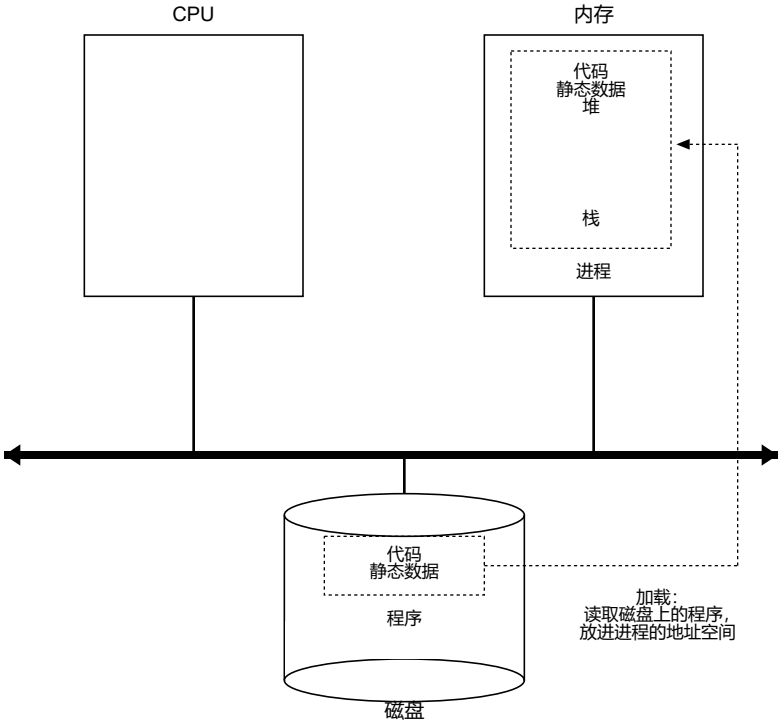


图 4.1：**加载：从程序到进程**

4.3 创建进程：再多讲一点

有一个谜团我们应该稍微揭开一下：程序是怎样变成进程的？具体来说，操作系统怎样把一个程序运行起来？进程创建实际上是怎么工作的？

要运行一个程序，操作系统首先要做的，是把它的代码和所有静态数据（例如已初始化的变量）**加载**（load）到内存中，放进进程的地址空间。程序最初以某种**可执行格式**（executable format）存放在**磁盘**（disk）上（在一些现代系统中，是**基于闪存的 SSD**，flash-based SSD）；因此，把程序和静态数据加载到内存的过程，需要操作系统从磁盘读出这些字节，放到内存中的某个地方（如图 4.1 所示）。

在早期的（或简单的）操作系统中，加载过程是**立即**（eagerly）完成的，也就是在运行程序之前一次性全部加载；现代操作系统则**惰性地**（lazily）执行这个过程，也就是只在程序执行期间需要某段代码或数据时，才加载那一部分。要真正理解代码和数据的惰性加载是如何工作的，你得更多地了解**分页**（paging）和**交换**（swapping）的机制，这些主题我们会在以后讨论内存虚拟化时讲到。现在只要记住：在运行任何东西之前，操作系统显然都得做一些工作，把程序的重要部分从磁盘弄到内存中。

代码和静态数据加载到内存之后，在运行进程之前，操作系统还需要做几件事。必须为程序的**运行时栈**（run-time stack，或者直接叫**栈**，stack）分配一些内存。你应该已经知道，C 程序用栈来存放局部变量、函数参数和返回地址；操作系统分配这块内存并交给进程。操作系统很可能还会用参数来初始化栈；具体来说，它会填好 `main()` 函数的参数，也就是 `argc` 和 `argv` 数组。

操作系统还可能为程序的**堆**（heap）分配一些内存。在 C 程序中，堆用来存放显式申请的动态分配数据；程序通过调用 `malloc()` 申请这类空间，通过调用 `free()` 显式释放。链表、散列表、树以及其他有趣的数据结构都需要用到堆。堆一开始会很小；程序运行时，会通过 `malloc()` 库 API 申请更多内存，这时操作系统可能会参与进来，为进程分配更多内存，以满足这些调用。

操作系统还会做一些其他的初始化工作，特别是与输入/输出（I/O）有关的工作。例如，在 UNIX 系统中，每个进程默认都打开了三个**文件描述符**（file descriptor），分别对应标准输入、标准输出和标准错误；有了这些描述符，程序就能方便地从终端读取输入，并把输出打印到屏幕上。我们会在本书关于**持久性**（persistence）的第三部分进一步学习 I/O、文件描述符之类的内容。

把代码和静态数据加载到内存，创建并初始化栈，再做一些与 I/O 设置有关的工作，至此，操作系统（终于）为程序的执行搭好了舞台。于是它还剩最后一项任务：从入口点，也就是 `main()`，开始运行程序。通过跳转到 `main()` 例程（借助我们下一章要讨论的一种专门机制），操作系统把 CPU 的控制权交给新创建的进程，程序就此开始执行。

4.4 进程状态

现在我们已经大致知道了进程是什么（尽管我们还会继续细化这个概念），以及（大致）它是怎样创建的，下面来谈谈进程在某一时刻可能处于的不同**状态**（state）。进程可以处于

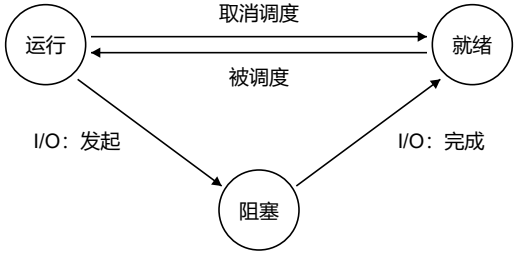


图 4.2：**进程：状态转换**

这些状态之一的概念，出现在早期的计算机系统中 [DV66, V+65]。简单来说，进程可以处于以下三种状态之一：

- **运行**（running）：在运行状态下，进程正在处理器上运行。这意味着它正在执行指令。
- **就绪**（ready）：在就绪状态下，进程已经准备好运行，但出于某种原因，操作系统选择此刻不运行它。
- **阻塞**（blocked）：在阻塞状态下，进程执行了某种操作，使它在发生其他某个事件之前都无法运行。一个常见的例子是：进程向磁盘发起 I/O 请求时，就会被阻塞，于是其他进程可以使用处理器。

如果把这些状态画成一张图，就会得到图 4.2 所示的图示。从图中可以看到，进程可以在操作系统的安排下，在就绪和运行状态之间来回移动。从就绪变为运行，意味着进程**被调度**（scheduled）了；从运行变为就绪，意味着进程被**取消调度**（descheduled）了。进程一旦被阻塞（例如因为发起了一次 I/O 操作），操作系统就会让它保持阻塞，直到某个事件发生（例如 I/O 完成）；这时，进程又回到就绪状态（如果操作系统这样决定，还可能马上再次进入运行状态）。

我们来看一个例子，看看两个进程可能怎样经历其中一些状态。首先，假设有两个进程在运行，它们都只使用 CPU（不做 I/O）。在这种情况下，每个进程的状态轨迹可能是这样的（图 4.3）。

在下一个例子中，第一个进程运行一段时间后发起了一次 I/O。这时，该进程被阻塞，让另一个进程有机会运行。图 4.4 展示了这个场景的轨迹。

时间	**进程$_0$**	**进程$_1$**	**说明**
1	运行	就绪	
2	运行	就绪	
3	运行	就绪	
4	运行	就绪	进程$_0$ 运行结束
5	–	运行	
6	–	运行	
7	–	运行	
8	–	运行	进程$_1$ 运行结束

图 4.3：**跟踪进程状态：只使用 CPU**

 © 2008–23, ARPACI-DUSSEAU

时间	进程$_0$	进程$_1$	说明
1	运行	就绪	
2	运行	就绪	
3	运行	就绪	进程$_0$ 发起 I/O
4	阻塞	运行	进程$_0$ 被阻塞,
5	阻塞	运行	于是 进程$_1$ 运行
6	阻塞	运行	
7	就绪	运行	I/O 完成
8	就绪	运行	进程$_1$ 运行结束
9	运行	–	
10	运行	–	进程$_0$ 运行结束

图 4.4：**跟踪进程状态：CPU 和 I/O**

更具体地说，进程$_0$ 发起一次 I/O，并在等待它完成时被阻塞；进程在从磁盘读取数据或者等待网络数据包时，就会被阻塞。操作系统发现 进程$_0$ 没有使用 CPU，就开始运行 进程$_1$。进程$_1$ 运行期间，I/O 完成了，进程$_0$ 重新回到就绪状态。最后，进程$_1$ 结束，进程$_0$ 运行，随后也结束了。

注意，即使在这个简单的例子中，操作系统也要做出许多决定。首先，在 进程$_0$ 发起 I/O 时，系统必须决定运行 进程$_1$；这样做让 CPU 保持忙碌，从而提高了资源利用率。其次，在 进程$_0$ 的 I/O 完成时，系统决定不切换回 进程$_0$；这是不是一个好决定还不清楚。你怎么看？这类决定由操作系统的**调度程序**（scheduler）做出，我们将在几章之后讨论这个主题。

4.5 数据结构

操作系统也是一个程序，和任何程序一样，它也有一些关键的数据结构，用来记录各种相关信息。例如，为了记录每个进程的状态，操作系统很可能会为所有就绪的进程维护某种**进程列表**（process list），并用一些额外的信息记录当前正在运行的是哪个进程。操作系统还必须以某种方式记录被阻塞的进程；当一个 I/O 事件完成时，操作系统应该确保唤醒正确的进程，让它再次准备好运行。

图 4.5 展示了 xv6 内核中操作系统需要为每个进程记录哪些信息 [CK+08]。在 Linux、Mac OS X 或 Windows 这样的“真正的”操作系统中，也有类似的进程结构；去查一查，看看它们要复杂多少。

从图中可以看到操作系统为进程记录的几项重要信息。对于一个已停止的进程，**寄存器上下文**（register context）保存着它的寄存器内容。进程停止时，它的寄存器会被保存到这块内存中；通过恢复这些寄存器（也就是把它们的值放回真正的物理寄存器），操作系统就能继续运行这个进程。我们会在以后的章节中进一步学习这种叫作**上下文切换**的技术。

从图中还可以看到，除了运行、就绪和阻塞之外，进程还可能处于其他一些状态。有时，系统会有一个**初始**（initial）状态，进程在创建过程中处于这个状态。此外，进程还可能被置于一个**最终**（final）状态：它已经退出，但还没有被清理（在基于 UNIX 的系统中，这称为**僵**

```
// xv6 为停止并在之后重新启动一个进程，
// 而保存和恢复的寄存器
struct context {
  int eip;
  int esp;
  int ebx;
  int ecx;
  int edx;
  int esi;
  int edi;
  int ebp;
};

// 进程可能处于的各种状态
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };

// xv6 为每个进程记录的信息，
// 包括它的寄存器上下文和状态
struct proc {
  char *mem;                  // 进程内存的起始地址
  uint sz;                    // 进程内存的大小
  char *kstack;               // 这个进程的
                              // 内核栈底
  enum proc_state state;      // 进程状态
  int pid;                    // 进程 ID
  struct proc *parent;        // 父进程
  void *chan;                 // 非零时，表示在 chan 上睡眠
  int killed;                 // 非零时，表示已被杀掉
  struct file *ofile[NOFILE]; // 打开的文件
  struct inode *cwd;          // 当前目录
  struct context context;     // 切换到这里来运行进程
  struct trapframe *tf;       // 当前中断的
                              // 陷阱帧
};
```

图 4.5：**xv6 的 proc 结构**

尸（zombie）状态[1]。这个最终状态很有用，因为它让其他进程（通常是创建这个进程的**父进程**，parent）能够检查该进程的返回码，看看刚结束的进程是否执行成功（在基于 UNIX 的系统中，程序成功完成任务时通常返回零，否则返回非零值）。完成后，父进程会做最后一次调用（例如 `wait()`），等待子进程结束，同时告诉操作系统，可以清理与这个已经消亡的进程有关的数据结构了。

[1]没错，僵尸状态。和真正的僵尸一样，这些僵尸相对容易杀死。不过，通常推荐使用的是不同的方法。

补充：数据结构——进程列表

操作系统里满是各种重要的**数据结构**（data structure），我们会在这些讲义中讨论它们。**进程列表**（process list，也叫**任务列表**，task list）是第一个这样的结构。它是比较简单的数据结构之一，但任何能够同时运行多个程序的操作系统，肯定都会有类似这样的结构，来记录系统中所有正在运行的程序。有时，人们把保存某个进程信息的单个结构称为**进程控制块**（Process Control Block，**PCB**），这只是一种花哨的说法，指的是一个包含每个进程信息的 C 结构体（有时也叫**进程描述符**，process descriptor）。

4.6 小结

我们介绍了操作系统最基本的抽象：进程。简单地说，它就是一个运行中的程序。有了这个概念上的认识，接下来我们就要进入具体细节：实现进程所需的底层机制，以及以智能的方式调度进程所需的高层策略。通过把机制和策略结合起来，我们将逐步理解操作系统是怎样虚拟化 CPU 的。

补充：进程的关键术语

- **进程**是操作系统对运行中的程序的主要抽象。在任何时刻，进程都可以用它的状态来描述：它的**地址空间**中内存的内容、CPU 寄存器的内容（包括**程序计数器**和**栈指针**等），以及有关 I/O 的信息（例如可以读写的已打开文件）。
- **进程 API** 由程序可以发出的、与进程有关的调用组成。通常包括创建、销毁以及其他有用的调用。
- 进程处于许多不同的**进程状态**之一，包括运行、就绪和阻塞。不同的事件（例如被调度或被取消调度，或者等待 I/O 完成）会使进程从一种状态转换到另一种状态。
- **进程列表**包含系统中所有进程的信息。每个条目都存放在有时被称为**进程控制块**（PCB）的东西中，它其实就是一个包含某个特定进程信息的结构体。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[BH70] “The Nucleus of a Multiprogramming System” by Per Brinch Hansen. Communications of the ACM, Volume 13:4, April 1970. *This paper introduces one of the first* **microkernels** *in operating systems history, called Nucleus. The idea of smaller, more minimal systems is a theme that rears its head repeatedly in OS history; it all began with Brinch Hansen's work described herein.*

[CK+08] “The xv6 Operating System” by Russ Cox, Frans Kaashoek, Robert Morris, Nickolai Zeldovich. From: https://github.com/mit-pdos/xv6-public. *The coolest real and little OS in the world. Download and play with it to learn more about the details of how operating systems actually work. We have been using an older version (2012-01-30-1-g1c41342) and hence some examples in the book may not match the latest in the source.*

[DV66] “Programming Semantics for Multiprogrammed Computations” by Jack B. Dennis, Earl C. Van Horn. Communications of the ACM, Volume 9, Number 3, March 1966. *This paper defined many of the early terms and concepts around building multiprogrammed systems.*

[L+75] “Policy/mechanism separation in Hydra” by R. Levin, E. Cohen, W. Corwin, F. Pollack, W. Wulf. SOSP ’75, Austin, Texas, November 1975. *An early paper about how to structure operating systems in a research OS known as Hydra. While Hydra never became a mainstream OS, some of its ideas influenced OS designers.*

[V+65] “Structure of the Multics Supervisor” by V.A. Vyssotsky, F. J. Corbato, R. M. Graham. Fall Joint Computer Conference, 1965. *An early paper on Multics, which described many of the basic ideas and terms that we find in modern systems. Some of the vision behind computing as a utility are finally being realized in modern cloud systems.*

 © 2008–23, ARPACI-DUSSEAU

作业（模拟）

程序 `process-run.py` 可以让你看到，当程序运行时，或者使用 CPU（例如执行一条加法指令），或者做 I/O（例如向磁盘发送请求并等待它完成），进程状态是如何变化的。详见 README。

问题

1. 用以下选项运行 `process-run.py`：`-l 5:100,5:100`。CPU 利用率（例如，CPU 处于使用状态的时间百分比）应该是多少？你为什么知道？用 `-c` 和 `-p` 选项看看你是否正确。
2. 现在用这些选项运行：`./process-run.py -l 4:100,1:0`。这些选项指定了一个有 4 条指令（全部使用 CPU）的进程，以及一个只发起一次 I/O 并等待它完成的进程。两个进程全部完成需要多长时间？用 `-c` 和 `-p` 看看你是否正确。
3. 交换进程的顺序：`-l 1:0,4:100`。现在会发生什么？交换顺序有影响吗？为什么？（和往常一样，用 `-c` 和 `-p` 看看你是否正确。）
4. 现在我们来研究其他一些选项。一个重要的选项是 `-S`，它决定了进程发起 I/O 时系统如何反应。当这个选项设为 `SWITCH_ON_END` 时，系统在一个进程做 I/O 期间不会切换到另一个进程，而是等到这个进程完全结束。运行下面两个进程（`-l 1:0,4:100 -c -S SWITCH_ON_END`），一个做 I/O，另一个做 CPU 计算，会发生什么？
5. 现在运行同样的进程，但把切换行为设为：每当一个进程在等待（WAITING）I/O 时，就切换到另一个进程（`-l 1:0,4:100 -c -S SWITCH_ON_IO`）。现在会发生什么？用 `-c` 和 `-p` 确认你是否正确。
6. 另一个重要的行为是 I/O 完成时该做什么。使用 `-I IO_RUN_LATER` 时，I/O 完成后，发起它的进程不一定马上运行；而是当时正在运行的进程继续运行。运行下面这组进程会发生什么？

   ```
   ./process-run.py -l 3:0,5:100,5:100,5:100 -S SWITCH_ON_IO
   -c -p -I IO_RUN_LATER
   ```

   系统资源得到有效利用了吗？
7. 现在运行同样的进程，但设置 `-I IO_RUN_IMMEDIATE`，这会立即运行发起 I/O 的那个进程。这种行为有什么不同？为什么让刚完成 I/O 的进程马上再次运行可能是个好主意？
8. 现在用一些随机生成的进程运行，选项为 `-l 3:50,3:50`，再分别加上 `-s 1`、`-s 2` 或 `-s 3`。看看你能否预测轨迹会是什么样子。分别使用 `-I IO_RUN_IMMEDIATE` 和 `-I IO_RUN_LATER` 时，会发生什么？分别使用 `-S SWITCH_ON_IO` 和 `-S SWITCH_ON_END` 时，又会发生什么？

 © 2008–23, ARPACI-DUSSEAU